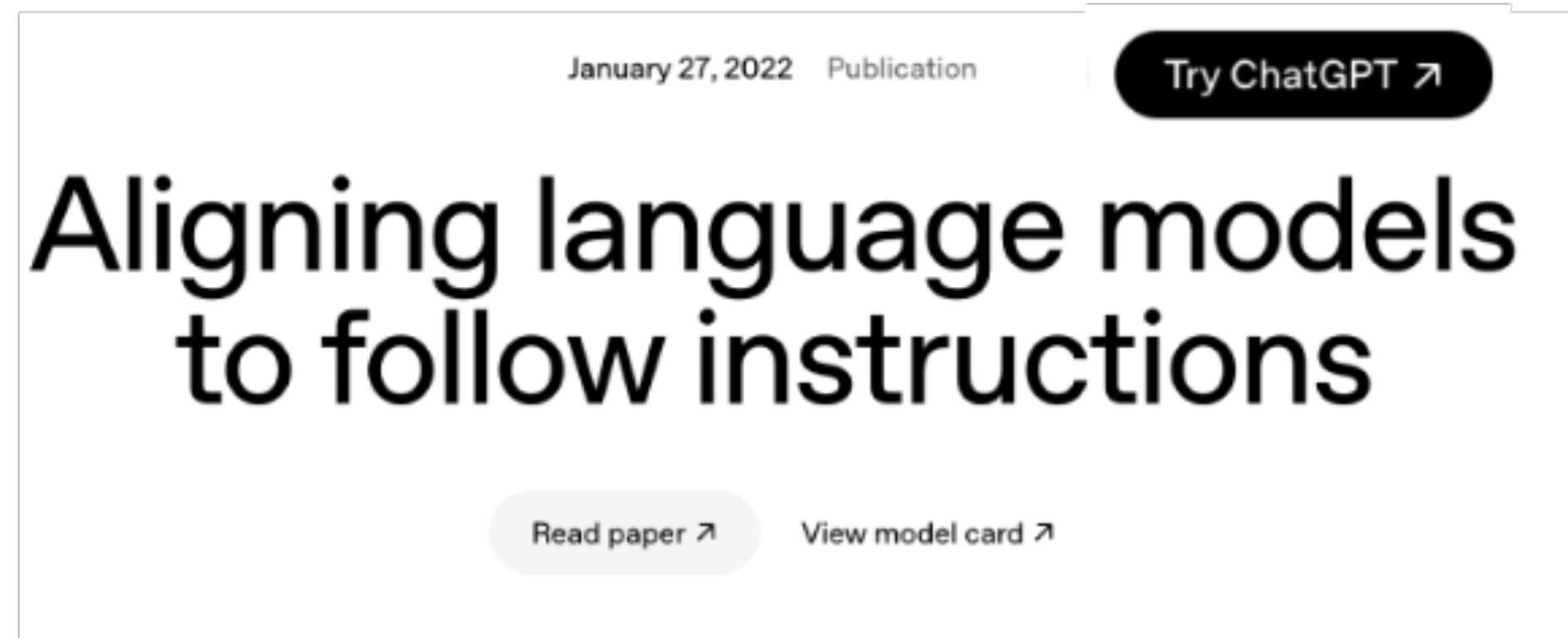


Direct Preference Optimization (DPO)

2022: RL from human feedback (RLHF)

→ Helpful assistants (ChatGPT)



OpenAI, NeurIPS'22

2025: RL from verifiers (RLVR)

→ Reasoning capabilities



DeepSeek, Nature'25

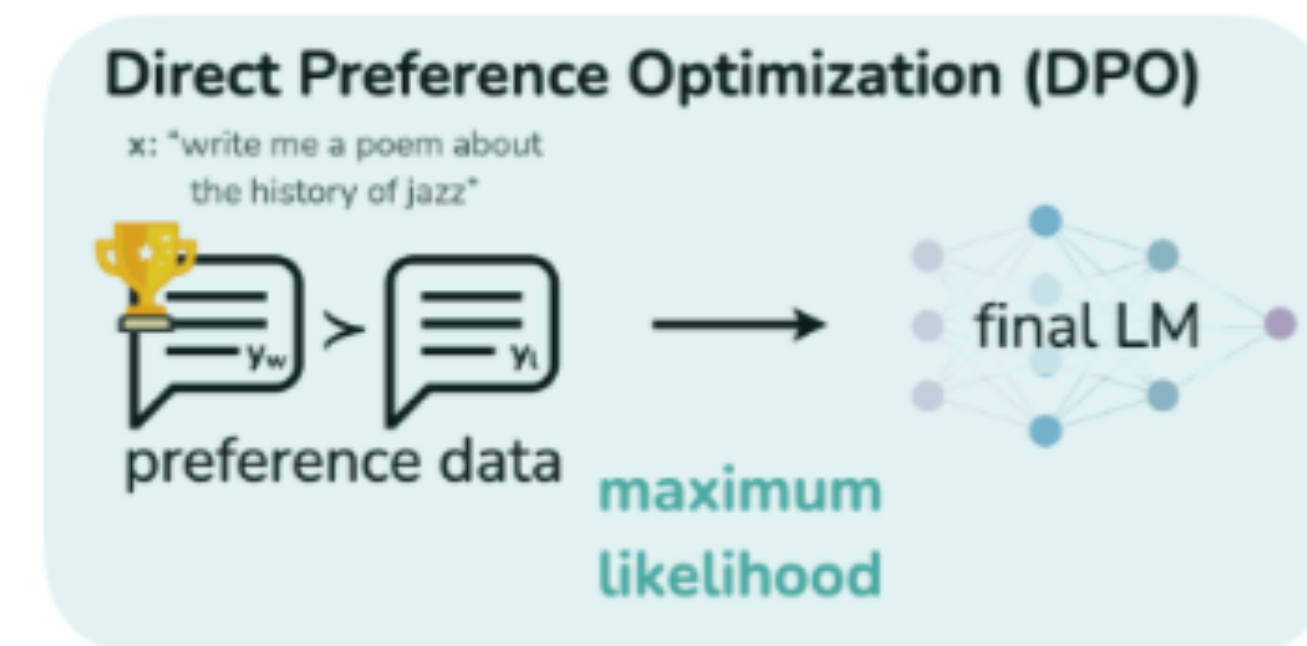
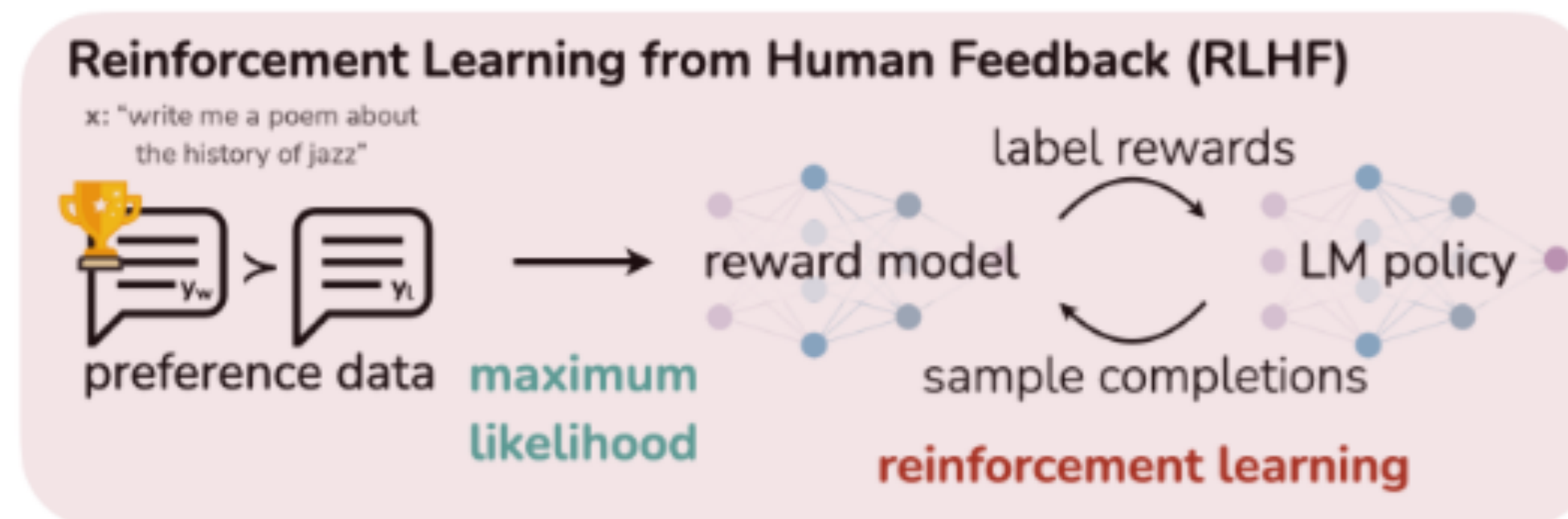
Two revolutions in reinforcement learning based post-training

DPO: Optimize From Preference Pairs

- Data: (x, y^+, y^-) , $y^+ \succ y^-$
- Choose a fixed reference π_{ref} (usually SFT)
- Train π_{θ} , by maximizing a preference log-likelihood

$$\max_{\theta} \mathbb{E} [\log P_{\theta}(y^+ \succ y^- \mid x)]$$

- Key move: parameterize $P_{\theta}(\cdot)$ using **policy log-prob ratios** (next slides)



Key Identity From The KL-Regularized Optimum

- From the previous theorem, for fixed x

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)$$

- Taking logs and rearranging gives:

$$r(x, y) = \beta \left(\log \pi^*(y \mid x) - \log \pi_{\text{ref}}(y \mid x) \right) + C(x)$$

where

$$C(x) = \beta \log Z(x) \quad \text{and} \quad Z(x) = \sum_{y'} \pi_{\text{ref}}(y' \mid x) \exp(r(x, y')/\beta).$$

Bradley–Terry Likelihood In Policy Log-Ratios

- Bradley–Terry model (Lecture 7):

$$P(y^+ \succ y^- \mid x) = \sigma(r(x, y^+) - r(x, y^-))$$

Substitute the difference form (dropping $C(x)$) :

$$r(x, y^+) - r(x, y^-) =$$

DPO Loss

- Given preference pairs (x, y^+, y^-) , define

$$u_{\theta}(x, y^+, y^-) = \beta \left[(\log \pi_{\theta}(y^+ | x) - \log \pi_{\theta}(y^- | x)) - (\log \pi_{\text{ref}}(y^+ | x) - \log \pi_{\text{ref}}(y^- | x)) \right].$$

- Then the DPO objective is maximum likelihood:

$$\max_{\theta} \mathbb{E} [\log \sigma(u_{\theta})]$$

- Equivalently, minimize the loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = - \mathbb{E}_{(x, y^+, y^-)} [\log \sigma(u_{\theta}(x, y^+, y^-))]$$

MLE On Preferences $\Rightarrow$ DPO Loss

- Model:

$$P_{\theta}(y^{+} \succ y^{-} \mid x) = \sigma(u_{\theta}(x, y^{+}, y^{-}))$$

- Log-likelihood over dataset $\mathcal{D}$:

$$\ell(\theta) = \sum_{(x, y^{+}, y^{-}) \in \mathcal{D}} \log \sigma(u_{\theta}(x, y^{+}, y^{-}))$$

- Negative log-likelihood:

$$-\ell(\theta) = \sum_{(x, y^{+}, y^{-}) \in \mathcal{D}} -\log \sigma(u_{\theta}(x, y^{+}, y^{-}))$$

What DPO Is Doing (Gradient View)

- For one pair, loss is $\ell(\theta) = -\log \sigma(u_\theta)$. Then,

$$\frac{\partial \ell}{\partial u_\theta} = -(1 - \sigma(u_\theta)).$$

And,

$$\frac{\partial u_\theta}{\partial \theta} = \beta \left(\nabla_\theta \log \pi_\theta(y^+ | x) - \nabla_\theta \log \pi_\theta(y^- | x) \right).$$

So

$$\nabla_\theta \ell(\theta) = -\beta(1 - \sigma(u_\theta)) \left(\nabla_\theta \log \pi_\theta(y^+ | x) - \nabla_\theta \log \pi_\theta(y^- | x) \right).$$

Role Of β in DPO

- Appears in the preference model:

$$P_{\theta}(y^{+} \succ y^{-} \mid x) = \sigma\left(\beta u_{\theta}(x, y^{+}, y^{-})\right)$$

- Appears in gradient magnitude:

$$\|\nabla_{\theta} \ell\| \propto \beta (1 - \sigma(u_{\theta}))$$

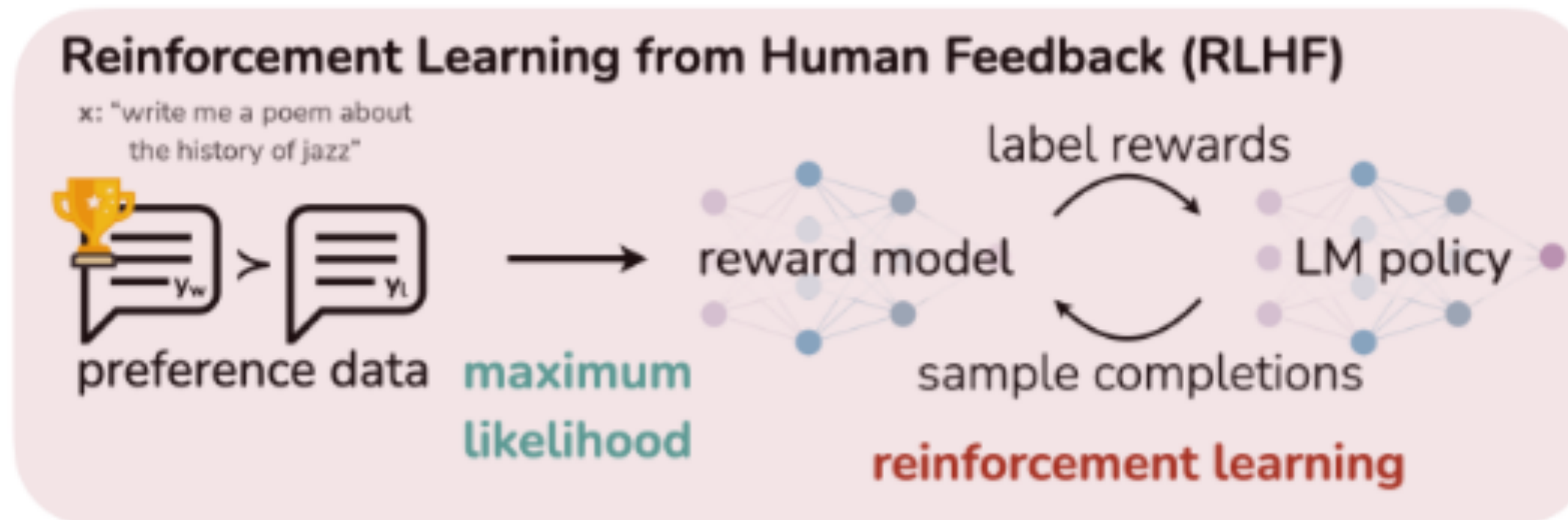
- Connection to KL-regularized optimum:

$$\pi^{*}(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta),$$

so β controls how strongly reward reshapes the reference distribution.

DPO Pipeline

- Inputs: preference pairs (x, y^+, y^-)
- Compute: $\log \pi_\theta(y^\pm | x)$ and $\log \pi_{\text{ref}}(y^\pm | x)$
- Optimize: $\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma(u_\theta)$



DPO Practical Checklist (What To Monitor)

- Monitor preference accuracy / win-rate on held-out pairs:

$$\mathbb{P}(u_{\theta}(x, y^+, y^-) > 0)$$

- Monitor KL drift from reference (token KL estimate):

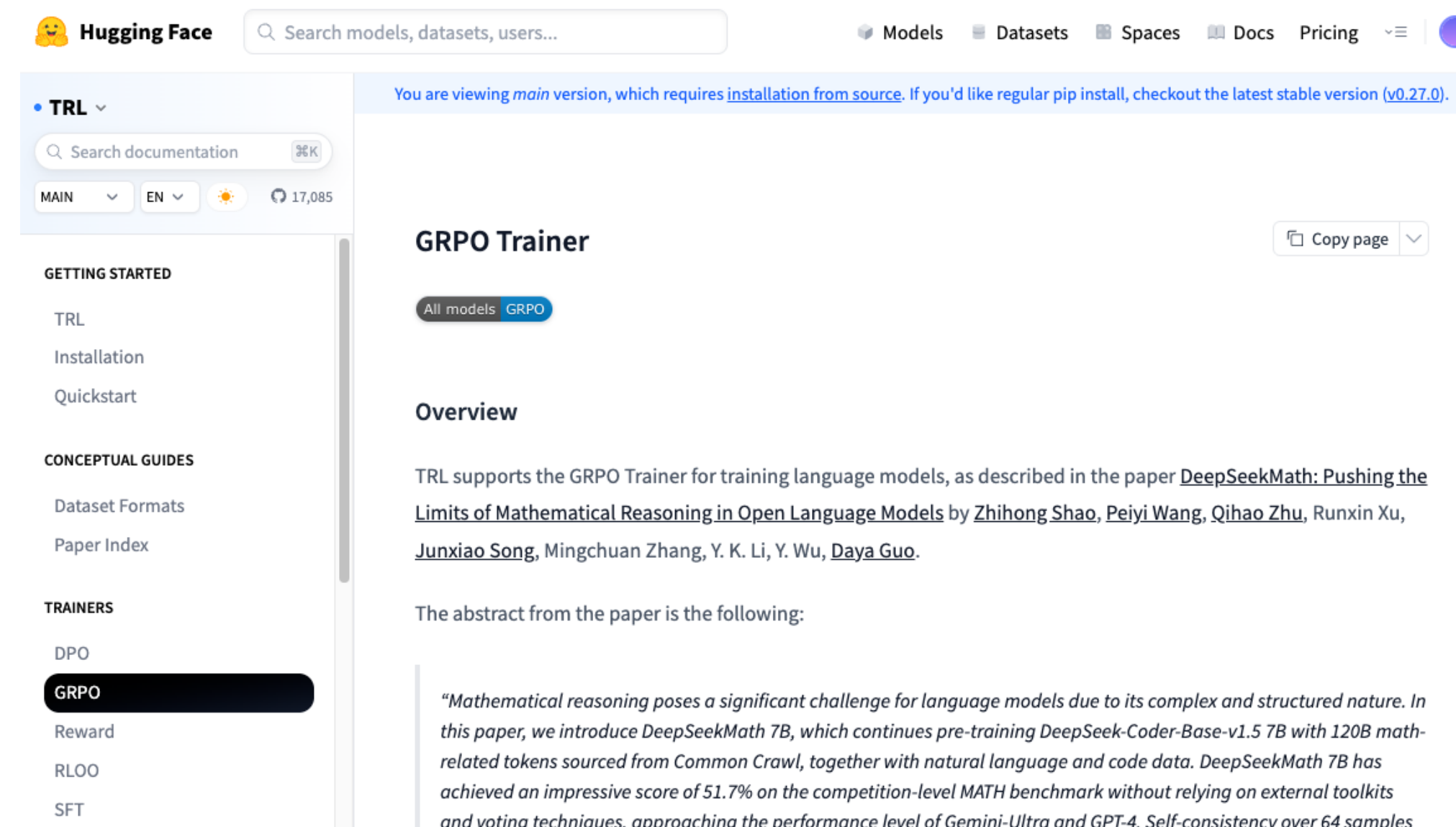
$$\widehat{\text{KL}} \approx \frac{1}{T} \sum_t (\log \pi_{\theta} - \log \pi_{\text{ref}})$$

- Watch length/verbosity drift (often correlates with log-prob shifts)
- Reference choice: π_{ref} fixed SFT vs moving reference (changes the target)

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization

- Online post-training: sample completions from the current policy, update π_θ
- For each prompt x , sample a group $\{y^{(i)}\}_{i=1}^G$, score each, and use group-relative advantages
- Optimize a PPO-like objective, optionally with KL-to-reference term
- Used in practice for reasoning-centric training (e.g., DeepSeekMath / DeepSeek-R1)



GRPO Training Loop

- Algorithm skeleton:

- 1) Sample prompts $x \sim \mathcal{D}$; generate $\{y^{(i)}\}_{i=1}^G \sim \pi_{\theta}(\cdot \mid x)$
- 2) Compute rewards $r^{(i)} = r(x, y^{(i)})$ (RM / verifier / reward function)
- 3) Compute group-relative advantages $A^{(i)}$ (next slide)
- 4) Update θ using GRPO loss (token-level sum; optional KL term)

(TRL) GRPOTrainer

```
# train_grpo.py
from datasets import load_dataset
from trl import GRPOTrainer
from trl.rewards import accuracy_reward
```

```
dataset = load_dataset("trl-lib/DeepMath-103K", split="train")

trainer = GRPOTrainer(
    model="Qwen/Qwen2-0.5B-Instruct",
    reward_funcs=accuracy_reward,
    train_dataset=dataset,
)
trainer.train()
```

[{ "content": "Find the distance from the origin to the curve defined by the equations $\left(\frac{x^2}{4} + y^2 + \frac{z^2}{4} \right) = 1$...		\$1\$
[{ "content": "Compute the limit: $\lim_{x \rightarrow 0} \frac{3x^2 - 3x \sin x}{x^2 + x \cos \frac{1}{x}}$ ", "role": "user" ...		\$0\$
[{ "content": "Let $p(x)$ be the polynomial of degree at most 8 such that $p(n) = \frac{1}{n}$ for $n = 1, 2, 3, \dots, 9$...		$\frac{1}{5}$
[{ "content": "Let A be a ring with 1 , possibly commutative. Consider the commutative monoid $\mathrm{Proj}(A)$ of isomorphism...		Yes

https://huggingface.co/docs/trl/main/en/grpo_trainer

Group-Relative Advantage

- For a fixed prompt x , sample $\{y^{(i)}\}_{i=1}^G$ and compute scores $r^{(i)} = r(x, y^{(i)})$
- Define (TRL default normalization):

$$\bar{r} = \frac{1}{G} \sum_{j=1}^G r^{(j)}, \quad s_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r^{(j)} - \bar{r})^2}$$

$$A^{(i)} = \frac{r^{(i)} - \bar{r}}{s_r}$$

and use the same $A^{(i)}$ across tokens in completion $y^{(i)}$

GRPO Vs PPO: What Changed

- **PPO (token-level):**

$$L^{\text{clip}}(\theta) = \mathbb{E} \left[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t) \right]$$

with A_t typically from a value baseline.

- **GRPO (group baseline):** for each completion i , use a sequence-level advantage $A^{(i)}$, shared across all its tokens:

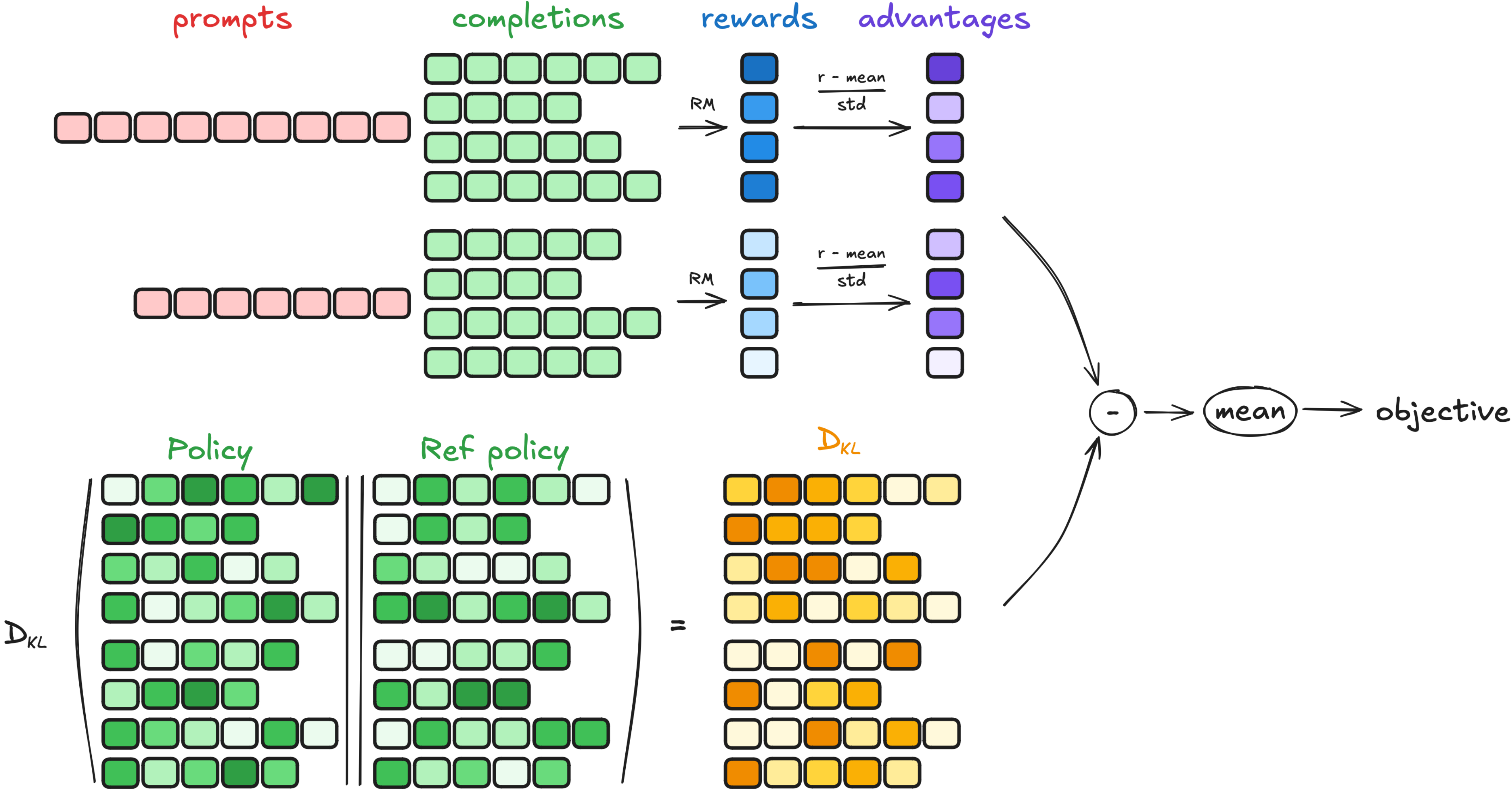
$$\ell_{i,t}(\theta) = r_{i,t}(\theta)A^{(i)} - \beta \widehat{\text{KL}} \left(\pi_{\theta} \parallel \pi_{\text{ref}} \right), \quad r_{i,t}(\theta) = \frac{\pi_{\theta}(y_t^{(i)} \mid x, y_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(y_t^{(i)} \mid x, y_{<t}^{(i)})}.$$

and sum $\sum_{i,t} \ell_{i,t}$

- If doing multiple updates per generation, GRPO uses the PPO-style clipped ratio form.

$$\mathcal{L}_{\text{GRPO}}(\theta) = - \frac{1}{\sum_{i=1}^G |y^{(i)}|} \sum_{i=1}^G \sum_{t=1}^{|y^{(i)}|} \left[\min \left(r_{i,t}(\theta)A^{(i)}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon \right) A^{(i)} \right) - \beta \widehat{\text{KL}} \left(\pi_{\theta} \parallel \pi_{\text{ref}} \right) \right]$$

GRPO Visual (TRL)



Why GRPO Foreshadows Reasoning RL

- Reasoning RL often uses verifier-style rewards (correct/incorrect, rubric, unit tests).
- Group sampling + relative advantage $\approx$ built-in within-prompt comparison, aligning naturally with reasoning datasets.
- Empirical connection: GRPO is used in DeepSeekMath and DeepSeek-R1 training pipelines.



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

For each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

TRL Ecosystem

- TRL provides training loops for post-training objectives:
 - **SFTTrainer** (supervised fine-tuning)
 - **DPOTrainer** (preference pairs)
 - **GRPOTrainer** (online group-relative updates)
- Built on top of Transformers + Accelerate;
- Common use with PEFT/LoRA
- Key idea: you implement an objective + data; TRL handles the loop + logging.



TRAINERS

DPO

GRPO

Reward

RLOO

SFT

HOW-TO GUIDES

Command Line Interface (CLI)

Training using Jobs

Customizing the Training

Reducing Memory Usage

Speeding Up Training

Distributing Training

Using Trained Models

TRL Minimal “Wiring Diagram” (What You Actually Need)

- **Inputs:**

model π_θ , reference π_{ref} , data $\mathcal{D}$

- **Data formats:**

- SFT: $(x, y^\star)$
- DPO: (x, y^+, y^-)
- GRPO: prompts x (+ reward function to score samples)

- **Core computation shared by all:**

$\log \pi_\theta(y \mid x)$ and often $\log \pi_{\text{ref}}(y \mid x)$

- **Optimization:** SGD/Adam on a scalar loss (your objective)

What To Log In Real Runs (Minimal Dashboard)

- Held-out preference win-rate (DPO-style):

$$\hat{p}_{\text{win}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{u_{\theta}(x_i, y_i^+, y_i^-) > 0\}$$

- KL-to-reference (token estimate):

$$\widehat{\text{KL}} = \frac{1}{B} \sum_{(x,y)} \frac{1}{T} \sum_{t=1}^T \left(\log \pi_{\theta}(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right)$$

- Reward proxy / score (RM/verifier metric): $\widehat{\mathbb{E}}[r(x, y)]$
- Length / verbosity: $\widehat{\mathbb{E}}[T]$

Demo A: Setup On A Finite Completion Set

- Fix a prompt x
- Choose a finite candidate set $\mathcal{Y} = \{y_1, \dots, y_G\}$
- Specify: $\pi_{\text{ref}}(y_i \mid x), \quad r_i := r(x, y_i)$
- Compute $\pi^*(\cdot \mid x)$ for multiple β

Demo A: Closed-Form Optimum

$$\pi^*(y_i | x) = \frac{\pi_{\text{ref}}(y_i | x) \exp(r_i/\beta)}{\sum_{j=1}^G \pi_{\text{ref}}(y_j | x) \exp(r_j/\beta)}$$

- Define the normalizer:

$$Z(x) = \sum_{j=1}^G \pi_{\text{ref}}(y_j | x) \exp(r_j/\beta)$$

- Equivalently:

$$\pi^*(\cdot | x) = \text{softmax}(\log \pi_{\text{ref}}(\cdot | x) + r(\cdot)/\beta)$$

Demo A: What We Measure

- Expected reward under the tilted policy:

$$\mathbb{E}_{\pi^*}[r] = \sum_{i=1}^G \pi^*(y_i | x) r_i$$

- KL drift from the reference:

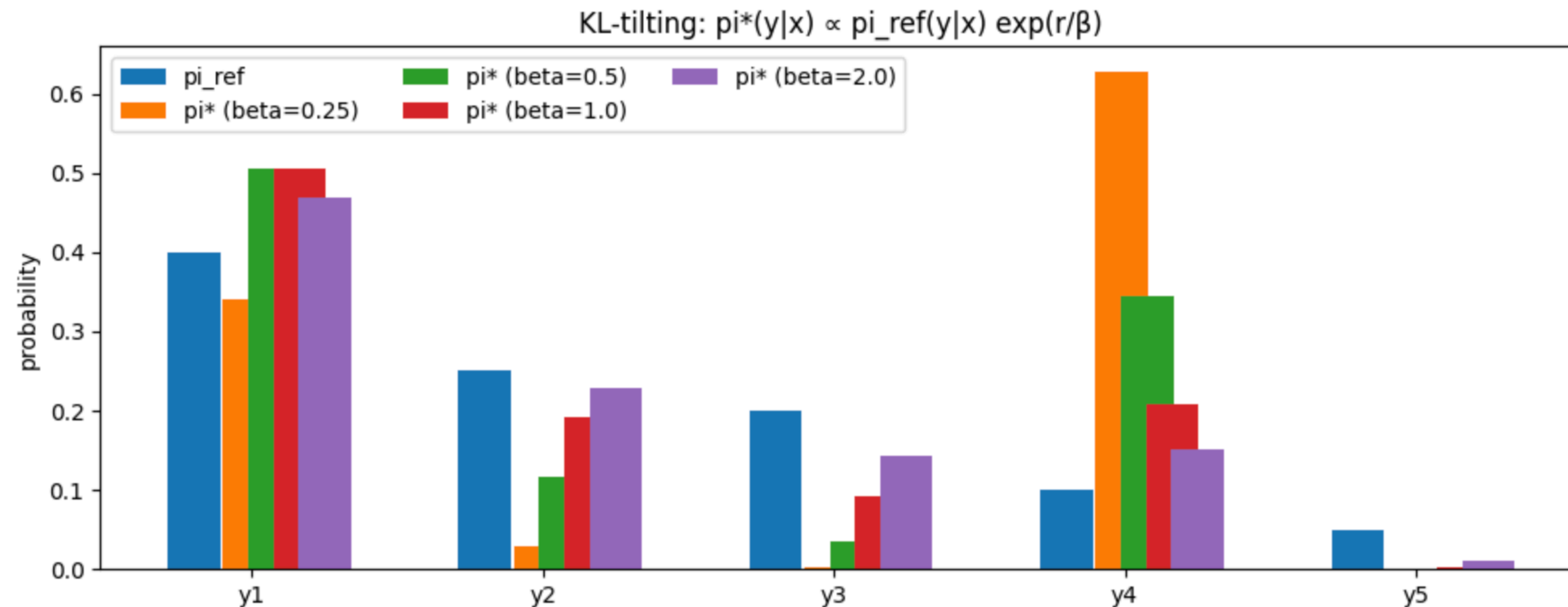
$$\text{KL}(\pi^* || \pi_{\text{ref}}) = \sum_{i=1}^G \pi^*(y_i | x) \log \frac{\pi^*(y_i | x)}{\pi_{\text{ref}}(y_i | x)}$$

- We will plot the **tradeoff curve**:

$$\beta \mapsto \left(\text{KL}(\pi^* || \pi_{\text{ref}}), \mathbb{E}_{\pi^*}[r] \right)$$

Demo A: Distribution Shift For Different β

- Plot $\pi_{\text{ref}}(y_i | x)$ and $\pi^*(y_i | x)$ for several β values: $\beta \in \{0.25, 0.5, 1.0, 2.0\}$
- Visual: bar chart over $i = 1, \dots, G$ (one bar per candidate completion)

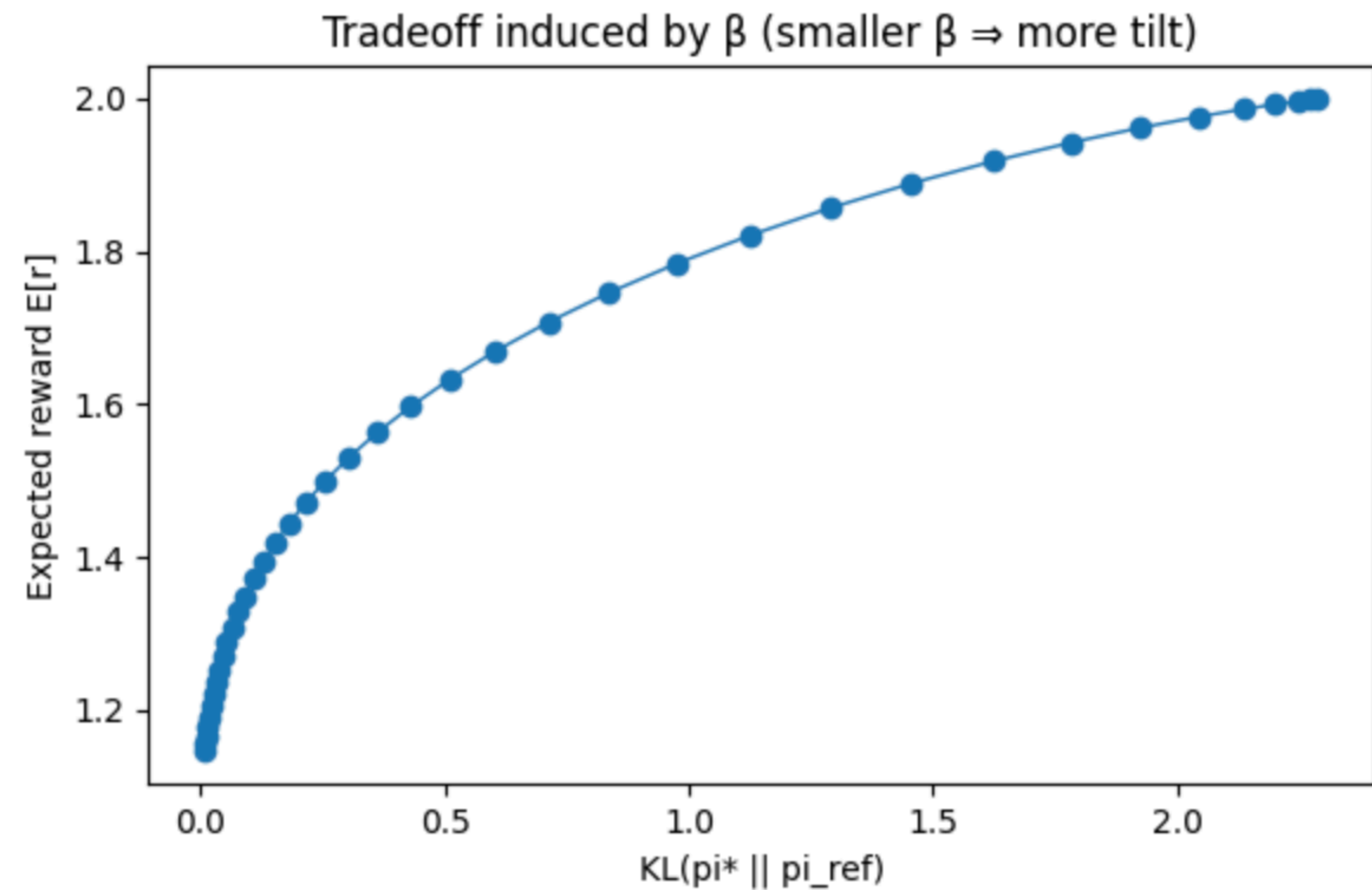


Demo A: Tradeoff Curve

For each $\beta > 0$, we compute: $\left(\text{KL}(\pi^* || \pi_{\text{ref}}), \mathbb{E}_{\pi^*}[r] \right)$

- Plot:

- x-axis: $\text{KL}(\pi^* || \pi_{\text{ref}})$
- y-axis: $\mathbb{E}_{\pi^*}[r]$



Demo A: KL Budget $\Rightarrow$ Choose β

- Constrained problem (per prompt x):
- $\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)}[r(x, y)] \quad \text{s.t.} \quad \text{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) \leq \varepsilon$
- In the demo: for each ε , find a β such that $\text{KL}(\pi_{\beta}^* \parallel \pi_{\text{ref}}) \leq \varepsilon$
- Output: “KL budget $\varepsilon \leftrightarrow$ conservative β ”

Takeaways

- Closed-form KL-regularized optimum:

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)$$

- β selects a point on the frontier: $\beta \downarrow: \mathbb{E}[r] \uparrow, \text{KL} \uparrow$ $\beta \uparrow: \mathbb{E}[r] \downarrow, \text{KL} \downarrow$
- RLHF methods differ mainly in how they approximate/optimize this idea:
 - PPO: sampled rollouts + clipped ratio + KL control
 - DPO: preference pairs + logistic loss + reference correction
 - GRPO: sampled groups + relative advantages + optional KL

Alignment

Alignment As Constrained RL / Multi-Objective RL

- Two reward signals (illustrative decomposition): $r_{\text{help}}(x, y), \quad r_{\text{harm}}(x, y)$

- Constrained formulation:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot|x)} [r_{\text{help}}(x, y)] \quad \text{s.t.} \quad \mathbb{E} [r_{\text{harm}}(x, y)] \leq \delta$$

- Lagrangian / scalarization:

$$\max_{\pi} \mathbb{E} [r_{\text{help}}(x, y) - \lambda r_{\text{harm}}(x, y)] \quad (\lambda \geq 0)$$

- With reference anchoring (post-training reality):

$$\max_{\pi} \mathbb{E} [r_{\text{help}} - \lambda r_{\text{harm}}] - \beta \mathbb{E} [\text{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))]$$

Where Alignment Enters The Pipeline

- Alignment enters via four interfaces:
 - **Data design**
prompts x , instructions, preference criteria, annotator guidelines
 - **Reward design**
reward model / verifier / rubric defines $r(x, y)$
 - **Constraints / filters**
hard constraints $y \notin \mathcal{Y}_{\text{blocked}}(x)$, refusal policies, safety classifiers
 - **Reference anchoring**
KL-to- π_{ref} controls drift from SFT
- A compact “system view”:

$$(\text{data}) \Rightarrow r(x, y) \Rightarrow \text{optimize } \pi_{\theta} \text{ subject to constraints} + \text{KL}$$

Failure Modes And Quantitative Evaluation

- **Over-optimization / reward hacking:** $\mathbb{E}_{\pi}[r] \uparrow \not\Rightarrow$ true utility $\uparrow$
- **Spec gaming / shortcut learning:** optimize proxy features correlated with r
- **Distribution shift:** $x \sim \mathcal{D}_{\text{train}} \neq x \sim \mathcal{D}_{\text{deploy}}$
- **KL collapse / drift (monitor):**

$$\widehat{\text{KL}}(\pi || \pi_{\text{ref}}), \quad \text{length drift,} \quad \text{toxicity proxy}$$

- Minimum evaluation set:
 - held-out preference win-rate / accuracy (DPO-style)
 - adversarial prompts / red-teaming set
 - calibration on safety constraints (violation rate)

Unification Map: Same KL Objective, Different Estimators

- Start from the KL-regularized target:

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$$

- Three ways to fit / approximate it:
 - PPO-RLHF: on-policy sampling + advantage estimation + clipping
 - DPO: offline preference pairs + logistic loss (implicit reward differences)
 - GRPO: on-policy groups + group-relative advantage (no value net)
- One-line summary:

same target family $\Rightarrow$ different data/estimators/loops

When Would You Use What?

- If you have **offline preference pairs** at scale:
 - DPO is simplest and stable (no on-policy rollouts)
- If you can do **on-policy sampling** and need fine control:
 - PPO-RLHF (heavier: value model, rollouts, KL control)
- If you have **verifier-style rewards** and want value-net-free training:
 - GRPO (group sampling; relative advantage baseline)
- Operational considerations:
 - compute budget (rollouts expensive)
 - reward reliability (RM vs verifier)
 - need for KL/constraint enforcement
 - evaluation (win-rate, drift, safety violation rate)

Reading Pointers

- InstructGPT: end-to-end RLHF pipeline (pretrain → SFT → RM → PPO)
- PPO: clipped objective and trust-region intuition (original PPO paper)
- DPO: direct preference optimization from pairs
- TRL docs: practical trainers (SFT/DPO/GRPO)
- Transformer architecture